Lab Exercise 8- API Security Threats (OWASP API Top 10) Using JAVA, Spring Boot+ Postman

---

1. Objective

To:

1. Build a **vulnerable REST API** demonstrating OWASP API Top 10 threats
2. Build a **secure version of the same API** using Spring Security
3. Test both vulnerable and secure APIs using **Postman**
4. Document and verify **security behavior**

---

2. Software / Tools Required

VS Code
Java JDK 17 / 21
Spring Boot 3.x / 4.x
Maven
Postman API Client
Swagger UI (for API documentation)

Recommended VS Code Extensions:

- Extension Pack for Java
- Spring Boot Extension Pack
- Maven for Java

---

3. OWASP API Top 10 Security Threats

ID	Threat	Description
API1	Broken Object Level Authorization (BOLA)	Attackers access other users' data
API2	Broken Authentication	Weak authentication mechanisms
API3	Broken Object Property Level Authorization)	Over-posting or mass assignment
API4	Unrestricted Resource Consumption	No rate limiting (DoS risk)
API5	Broken Function Level Authorization	Unauthorized admin actions
API6	Unrestricted Access to Business Flow	Sensitive workflows not protected
API7	Server Side Request Forgery (SSRF)	API fetches attacker-controlled URLs
API8	Security Misconfiguration	Debug endpoints exposed
API9	Improper Inventory Management	Unmanaged API versions
API10	Unsafe Consumption of APIs	Trusting unverified external APIs

---

4. Create Spring Boot Project in VS Code

Step 4.1 Open VS Code

Press:

```
Ctrl + Shift + P
```

Select:

Spring Initializr: Generate Maven Project

Step 4.2 Project Metadata

Project: **Maven**

Language: **Java**

Group: **com.example**

Artifact: **securityapi**

Name: **securityapi**

Packaging: **Jar**

Java Version:17

Step 4.3 Dependencies

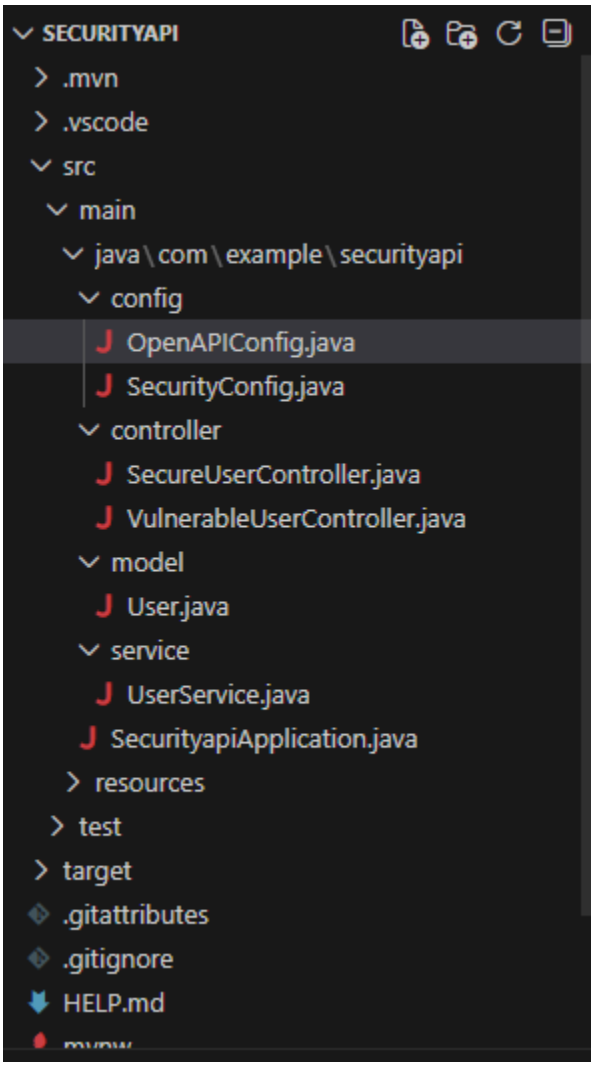
Select:

Spring Web

Then open the project in **VS Code**.

---

5. Project Structure



---

6. Maven Dependency (pom.xml)

Verify dependencies in **pom.xml**

```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
<groupId>org.springdoc</groupId>
<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
<version>2.5.0</version>
</dependency>
```

---

1. Main Application File

SecurityApiApplication.java

Path:

src/main/java/com/example/securityapi/SecurityApiApplication.java

```
package com.example.securityapi;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SecurityApiApplication {
   public static void main(String[] args) {
      SpringApplication.run(SecurityApiApplication.class, args);
   }
}
```

---

2. Model Class

User.java

Path:

model/User.java

```
package com.example.securityapi.model;

import com.fasterxml.jackson.annotation.JsonIgnore;

public class User {

   private Long id;
   private String name;
   private String email;
```

```
private String role;

@JsonIgnore
private String password;

public User() {}

public User(Long id, String name, String email, String role, String password) {
   this.id = id;
   this.name = name;
   this.email = email;
   this.role = role;
   this.password = password;
}

public Long getId() {
   return id;
}

public String getName() {
   return name;
}

public String getEmail() {
   return email;
}

public String getRole() {
   return role;
}

@JsonIgnore
```

```
    public String getPassword() {
        return password;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public void setRole(String role) {
        this.role = role;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

---

3. Service Layer

UserService.java

Path:

service/UserService.java

```
package com.example.securityapi.service;


import com.example.securityapi.model.User;
import org.springframework.stereotype.Service;


import java.util.Collection;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;


@Service
public class UserService {


   private final Map<Long, User> store = new HashMap<>();
   private final AtomicLong counter = new AtomicLong(3);


   public UserService() {


      store.put(1L,
            new User(
                  1L,
                  "Admin",
                  "admin@test.com",
                  "admin",
```

```
                "admin123"
        ));

    store.put(2L,
        new User(
                2L,
                "User",
                "user@test.com",
                "user",
                "user123"
        ));
}

public User find(Long id) {
    return store.get(id);
}

public Collection<User> findAll() {
    return store.values();
}

public User save(User user) {

    if (user.getId() == null) {
```

```
        user.setId(counter.getAndIncrement());
      }

      store.put(user.getId(), user);

      return user;
   }

   public  boolean delete(Long id) {

      return store.remove(id) != null;

   }
}
```

---

5. Vulnerable Controller

VulnerableUserController.java

Path:

controller/VulnerableUserController.java

```
package com.example.securityapi.controller;

import com.example.securityapi.model.User;
import com.example.securityapi.service.UserService;
import org.springframework.web.bind.annotation.*;
```

```
import java.util.Collection;

@RestController
@RequestMapping("/api/v1")
public class VulnerableUserController {

   private final UserService service;

   public VulnerableUserController(UserService service) {
      this.service = service;
   }

   // API1 – Broken Object Level Authorization (BOLA)
   @GetMapping("/users/{id}")
   public User getUser(@PathVariable Long id) {

      return service.find(id);

   }

   // API3 – Mass Assignment
   @PostMapping("/users")
   public User createUser(@RequestBody User user) {

      return service.save(user);

   }

   // API2 – Broken Authentication
   @PostMapping("/login")
   public String login(@RequestBody User user) {
```

```
        User storedUser = service.find(user.getId());

        if (storedUser != null &&
                storedUser.getPassword().equals(user.getPassword())) {

            return "Login Successful!";
        }

        return "Invalid Login!";
    }

    // API5 – Broken Function Level Authorization
    @DeleteMapping("/admin/delete/{id}")
    public String deleteUser(@PathVariable Long id) {

        service.delete(id);

        return "User deleted!";

    }

    // API8 – Excessive Data Exposure
    @GetMapping("/debug/info")
    public Collection<User> debug() {

        return service.findAll();

    }
}
```

5. Secure Controller

SecureUserController.java

Path:

controller/SecureUserController.java

```
package com.example.securityapi.controller;

import com.example.securityapi.model.User;
import com.example.securityapi.service.UserService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/secure")
public class SecureUserController {

   private final UserService service;

   public SecureUserController(UserService service) {
      this.service = service;
   }

   @GetMapping("/users/{id}")
   @PreAuthorize("hasRole('ADMIN')")
   public ResponseEntity<User> getUser(@PathVariable Long id) {

      User user = service.find(id);

      if (user == null) {
```

```
            return ResponseEntity.notFound().build();

        }

        return ResponseEntity.ok(user);
    }

    @PostMapping("/users")
    public ResponseEntity<User> createUser(@RequestBody User user) {

        user.setRole("user");
        user.setId(null);

        return ResponseEntity.ok(service.save(user));

    }


    @DeleteMapping("/admin/delete/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public String deleteUser(@PathVariable Long id) {

        service.delete(id);

        return "User deleted successfully";

    }
}
```

6. Security Configuration

SecurityConfig.java

Path:

config/SecurityConfig.java

```
package com.example.securityapi.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSe
curity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;

import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;

import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

   @Bean
   public UserDetailsService userDetailsService() {

      UserDetails admin = User
```

```
            .withUsername("admin")
            .password("{noop}admin123")
            .roles("ADMIN")
            .build();

        UserDetails user = User
            .withUsername("user")
            .password("{noop}user123")
            .roles("USER")
            .build();

        return new InMemoryUserDetailsManager(admin, user);
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/**").permitAll()
                .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                .requestMatchers("/api/secure/**").authenticated()
            )
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}
```

---

7. Swagger Configuration

OpenAPIConfig.java

Path:

config/OpenAPIConfig.java

```
package com.example.securityapi.config;

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.Info;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class OpenAPIConfig {

   @Bean
   public OpenAPI customOpenAPI() {

      return new OpenAPI()
            .info(
                  new Info()
                        .title("OWASP API Security Lab")
                        .version("1.0")
                        .description("Demonstration of OWASP API Top 10 vulnerabilities")
            );
   }
}
```

---

8. application.properties

Path:

src/main/resources/application.properties

```
server.port=8080
```

---

9. Run the Project

In VS Code terminal:

```
mvn spring-boot:run
```

Open Swagger

```
http://localhost:8080/swagger-ui/index.html
```

---

14. Testing APIs using Postman

14.1 Vulnerable APIs (No Authentication)

BOLA Attack

GET

```
http://localhost:8080/api/v1/users/1
```

Mass Assignment Attack

POST

```
http://localhost:8080/api/v1/users
```

Body:

```
{
"id":99,
"name":"Hacker",
"role":"admin",
"email":"hack@test.com",
"password":"root"
}
```

Delete User Without Authorization

DELETE

```
http://localhost:8080/api/v1/admin/delete/1
```

Debug Information Exposure

GET

```
http://localhost:8080/api/v1/debug/info
```

14.2 Secure APIs (Basic Authentication)

In Postman:

Authorization → Basic Auth

Username

admin

Password

admin123

Secure Get User

GET

```
http://localhost:8080/api/secure/users/1
```

Secure Create User

POST

```
http://localhost:8080/api/secure/users
```

Secure Delete User

DELETE

http://localhost:8080/api/secure/admin/delete/2

15. Result

Thus, **OWASP API Top 10 security threats were successfully demonstrated** using a **vulnerable REST API** and mitigated using **Spring Security mechanisms**. All APIs were tested using **Postman API Client in VS Code environment**.